

Deploying the Fraud Detection Engine

Suyash Vishwakarma · deployed September 25–26, 2026

Summary

On September 25–26, 2026 I ran the full streaming fraud engine on one Hetzner Cloud server for about a day, then tore it down. I did it to practise a real deployment end to end, not to serve users.

Server	Hetzner CPX42: 8 AMD vCPU, 16 GB RAM, 320 GB disk, Helsinki, Ubuntu 26.04
Public endpoints	<code>https://api.suyash.fyi</code> (API key) and <code>https://grafana.suyash.fyi</code> (login), both behind Caddy with Let's Encrypt TLS
Private services	Kafka, Zookeeper, Redis, MLflow, Postgres, MinIO, Prometheus, kafka-ui, the scorer
Model served	<code>fraud-xgboost v6</code> via the <code>champion</code> alias: 439 features, test AUC 0.8932, threshold 0.7933
Proof	One transaction sent over public HTTPS was scored and read back; then 5,000 dataset rows were replayed through Kafka
Cost	About \$3.20 of server time for ~24 hours, plus \$5.20 for the domain

The self-contained demo console on Fly.io is a separate deployment and was not touched. This document explains what I did and why. The exact commands live in [DEPLOY.md](#); the model audit lives in [PROJECT_ANALYSIS.md](#).

How the engine works

Every transaction is queued in Kafka, scored by an XGBoost model pulled from MLflow, and stored in Redis for the API to serve. The dataset is IEEE-CIS: 590,540 card transactions, 3.5% fraud.

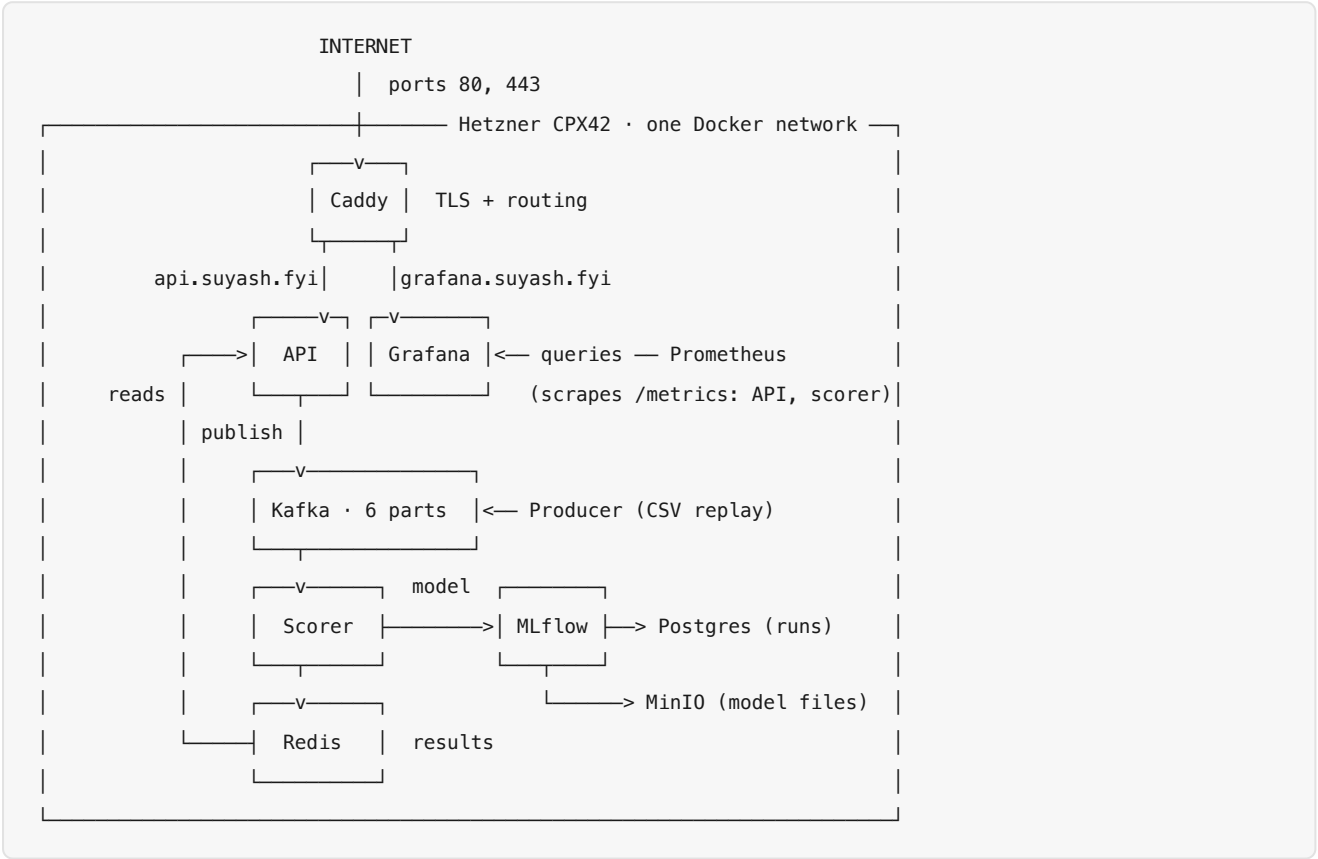
1. **Ingest.** Transactions enter Kafka in one of two ways: `producer.py` replays the CSV, or a client calls `POST /transactions` on the API. The topic has 6 partitions, keyed by `TransactionID`.
2. **Score.** The consumer (`consumer.py`, group `fraud-scorer`) reads in micro-batches, builds 439 features and calls the model once per batch. Bad messages go to a `transactions-dlq` topic instead of stopping the stream.
3. **Store.** Each result becomes a `prediction:<id>` key in Redis with a 1-hour expiry. Flagged transactions also go into a sorted set and `stats:*` counters. The consumer commits Kafka offsets only after Redis has the write, so a crash re-scores work rather than losing it.
4. **Serve.** The API (`API/main.py`) reads Redis for `/stats`, `/frauds` and `/transaction/{id}`. Every endpoint except `/health` and `/metrics` needs an `X-API-Key` header.
5. **Watch.** Prometheus scrapes `/metrics` from the consumer and the API, and Grafana charts Prometheus. They measure the system's health (throughput, latency, failures), not the fraud results, which stay in Redis.

The train/serve contract. Training writes four files next to the model in its MLflow run, under `preprocessing/`: `cat_maps.json`, `feature_columns.json`, `encoding_config.json` and `threshold_config.json`. At startup the consumer resolves `models:/fraud-xgboost@champion` and downloads those files from the same run, so the encoding can never drift from the model that uses it. A parity test asserts that the training path and the serving path build identical feature vectors.

Promotion. Registering a model does not deploy it. I promote a version by moving the `champion` alias, with an AUC gate in `scripts/promote_model.py`. The threshold 0.7933 was chosen on the validation split; at 0.5, three of every four alerts were false alarms.

Architecture on the server

The stack ran as 12 long-running containers plus two one-time setup jobs, all on one server and one private Docker network. Only Caddy listened on the internet.



Service	Reachable from	How
Caddy (API, Grafana)	anyone	HTTPS on 443; port 80 only redirects and answers Let's Encrypt
SSH	anyone holding my key	port 22
MLflow, Prometheus, kafka-ui, RedisInsight, MinIO console	me only	bound to <code>127.0.0.1</code> , reached over an SSH tunnel
Kafka, Redis, Postgres, scorer	other containers only	the Docker network, by container name

Two layers of protection. The Hetzner Cloud Firewall allows only ports 22, 80 and 443. Behind it, every internal service binds to `127.0.0.1` through `BIND_ADDRESS` in `.env`, so a firewall mistake would still not expose them. I used the cloud firewall instead of Ubuntu's `ufw` because Docker writes its own network rules that bypass `ufw`.

Why two public domains at all. The services find each other by container name and need no domain. I made the API public because a real fraud system's callers, such as a payment gateway, sit outside the server. I made Grafana public so I could watch it in a browser without a tunnel. Both are guarded: the API by a key, Grafana by a login. It also meant practising DNS, a reverse proxy and automatic TLS, which was the point of the exercise.

Deployment walkthrough

Each step lists what I did and why.

#	Step	What I did	Why
1	Domain	Bought <code>suyash.fyi</code> on Cloudflare for \$5.20/year, renewal at the same price	Let's Encrypt certificates and Caddy's routing both work by name. I skipped the \$4.18 <code>.bid</code> because that ending is spam-heavy and often blocked by company networks.
2	SSH key	Made a dedicated <code>ed25519</code> key with a passphrase and added only its public half to Hetzner	One key per purpose: if it leaks, I replace one key, not my GitHub access too
3	Firewall	Hetzner Cloud Firewall, inbound TCP 22, 80, 443 only	Everything else stays invisible; the cloud firewall sits outside the server, where Docker cannot bypass it
4	Server	CPX42 in Helsinki, x86, public IPv4 <code>2.29.58.145</code>	The compose memory limits add up to about 8.6 GB, so 16 GB leaves headroom; x86 avoids images without ARM builds
5	DNS	A records <code>api</code> and <code>grafana</code> → <code>2.29.58.145</code> , set to "DNS only"	With Cloudflare's proxy on, Cloudflare would terminate TLS instead of Caddy
6	Swap	Added 4 GB of swap	A memory spike while Kafka and the model load together should slow the box down, not kill a service
7	Docker	Installed with <code>get.docker.com</code>	Compose runs the same stack I run locally
8	Code	Cloned the public repo into <code>~/fraud-detect-system</code>	Compose names volumes after the folder, so matching the local name keeps every runbook command valid
9	Secrets	Wrote <code>.env</code> with <code>openssl rand -hex</code> passwords, <code>BIND_ADDRESS=127.0.0.1</code> , <code>API_BIND_ADDRESS=127.0.0.1</code> , the domain and <code>COMPOSE_FILE</code>	Compose refuses to start without the secrets instead of using weak defaults; <code>COMPOSE_FILE</code> loads the production overlay without typing <code>-f</code> twice
10	Start	<code>docker compose up -d --build</code>	Builds the pipeline and MLflow images, pulls the rest, starts everything in dependency order
11	Registry	Served my Mac's old MLflow store read-only on port 5002, tunnelled the server's MLflow to port 5555, and ran <code>scripts/migrate_mlflow_store.py</code>	The server's registry started empty, so the scorer had no <code>champion</code> to load. The script copied 26 runs and versions v1–v6, kept the version numbers, and set <code>champion</code> → v6.

#	Step	What I did	Why
12	Traffic	Sent one transaction over HTTPS, then <code>rsync</code> ed the two training CSVs (about 710 MB) and replayed 5,000 rows with the producer	The single request proves every hop; the replay makes the dashboards and consumer lag meaningful

What I skipped on purpose. DEPLOY.md also creates a non-root `deploy` user, turns on automatic security updates and disables SSH password login. I skipped them for a server that lived one day. Hetzner sets no root password when a server is created with an SSH key, so there was no password to brute-force. For anything longer-lived I would do all three.

Problems I hit and how I fixed them

Problem	Cause	Fix
<code>docker compose up</code> failed before starting anything	MinIO stopped publishing its image: <code>quay.io/minio/minio</code> returned 401 and <code>minio/minio</code> on Docker Hub returned 404. The other images only showed "Interrupted" because one failure aborted the whole pull.	Switched both MinIO services to <code>chainguard/minio:latest-dev</code> , a maintained build of the same server. It has no <code>curl</code> , so the healthcheck now uses <code>wget</code> . I tested server start, healthcheck and bucket creation in isolated containers on my Mac first, then patched the server with <code>sed</code> .
DEPLOY.md's hardening step would have locked <code>sudo</code>	It created <code>deploy</code> with no password, so <code>sudo</code> had nothing to check	Added <code>passwd deploy</code> ; the password is used only by <code>sudo</code> , SSH stays key-only
DEPLOY.md's "keys only" step might have left password login on	It edited <code>sshd_config</code> , but <code>sshd</code> keeps the first value it reads and cloud-init's <code>50-cloud-init.conf</code> is read first	Write a drop-in, <code>/etc/ssh/sshd_config.d/00-hardening.conf</code> , which is read before cloud-init's
Ports 80 and 443 refused connections after <code>up</code>	Caddy is created last, after the image build finished	Waited for the build; both ports opened and the certificate arrived within a minute
<code>git push</code> failed with <code>Permission denied (publickey)</code>	My GitHub key has a non-default name and there is no SSH config, so SSH offered GitHub nothing	Push through GitHub Desktop, or <code>GIT_SSH_COMMAND="ssh -i ~/.ssh/id_ed25519_personal" git push</code>
<code>rsync</code> tried to connect to itself	I ran the upload in the server terminal instead of on my Mac	Cancelled it and ran it from the Mac. The prompt tells the machines apart: <code>apple@MacBook-Pro</code> versus <code>root@ubuntu-16gb-hell-1</code> .
<code>/health</code> said <code>"kafka": "unknown"</code>	The API reports the result of its last publish, and nothing had been published yet	Not a fault; it turned <code>up</code> after the first transaction

Verification

I checked each layer from outside the server, from my Mac.

Check	Result
<code>https://api.suyash.fyi/health</code>	<code>200 {"status":"ok","redis":"up"}</code>
TLS certificate	Let's Encrypt, CN= <code>api.suyash.fyi</code> , valid until Dec 24, 2026

Check	Result
Plain <code>http://</code>	308 redirect to HTTPS
<code>/stats</code> without a key	401
<code>/metrics</code> through Caddy	404, blocked on purpose; Prometheus scrapes it inside the network
<code>https://grafana.suyash.fyi</code>	redirects to <code>/login</code>
Ports 5001, 6379, 9092, 8080, 8200, 9000, 9090, 3000	all closed from the internet
Registry through the tunnel	<code>champion</code> → v6, READY; all four <code>preprocessing/</code> files present; the model loads as <code>XGBClassifier</code> with 439 features
Scorer	joined group <code>fraud-scorer</code> and took all 6 partitions
One transaction over HTTPS	accepted, scored 0.0051 (LOW), <code>true_label</code> stripped, <code>/stats total_scored: 1</code>
Replay	5,000 rows at about 85 tx/s, joined with the identity table (144,233 rows)

The first transaction took 144 ms because the scorer was still warming up. Batched scoring of the replay runs far faster per transaction.

Cost and teardown

Item	Rate	For ~24 hours
CPX42 server	\$0.131/hour	about \$3.15
Public IPv4	about \$0.001/hour	about \$0.02
Traffic	20 TB included	\$0
Domain <code>suyash.fyi</code>	\$5.20/year, paid once	\$5.20

Teardown: delete the server in the Hetzner console, not just power it off; a stopped server still bills. Then check Primary IPs and Snapshots for leftovers. My Mac still holds the original model store, and everything on the server can be rebuilt from the repo and `DEPLOY.md`.

What this deployment is not

- **One host, one broker.** Kafka runs a single broker with replication factor 1 in Zookeeper mode. Losing the server loses undelivered messages.
- **Plaintext inside the network.** Kafka and Redis have no TLS between containers. The boundary is the firewall plus `127.0.0.1`, not authentication.
- **No alerting and no provisioned dashboards.** Prometheus collects and Grafana has its datasource, but nothing pages anyone and the dashboards are built by hand.
- **Ingestion is not idempotent.** Re-sending a `TransactionID` re-scores it and double-counts `stats:*`.
- **An unpinned image.** `chainguard/minio:latest-dev` moves with every release.

If I ran this for longer, I would pin every image to a digest, do the full hardening, provision Grafana dashboards and alerts, and script the server setup with cloud-init or Terraform, so a rebuild is one command instead of this walkthrough.

Screenshots and video

The live system is gone, so these captures from September 25–26, 2026 are the evidence. Files go in `docs/deploy/`.

Capture	File
Padlock and certificate on <code>api.suyash.fyi</code>	<code>docs/deploy/tls.png</code>
<code>/stats</code> after the replay	<code>docs/deploy/stats.png</code>
Grafana with live throughput	<code>docs/deploy/grafana.png</code>
MLflow model page, <code>champion</code> → v6	<code>docs/deploy/mlflow.png</code>
Consumer group lag near zero	<code>docs/deploy/consumer-lag.png</code>
Walkthrough video	link to be added